

- **Atomicity** - A transaction is an atomic unit of processing; it should either be performed in its entirety or not performed at all.

T_1
Read(A)
$A = A - 100$
Write(A)
Read(B)
$B = B + 100$
Write(B)

- Transactions should possess several properties, often called the **ACID** properties; to provide integrity and consistency of the data in the database. The following are the ACID properties:

A = Atomicity

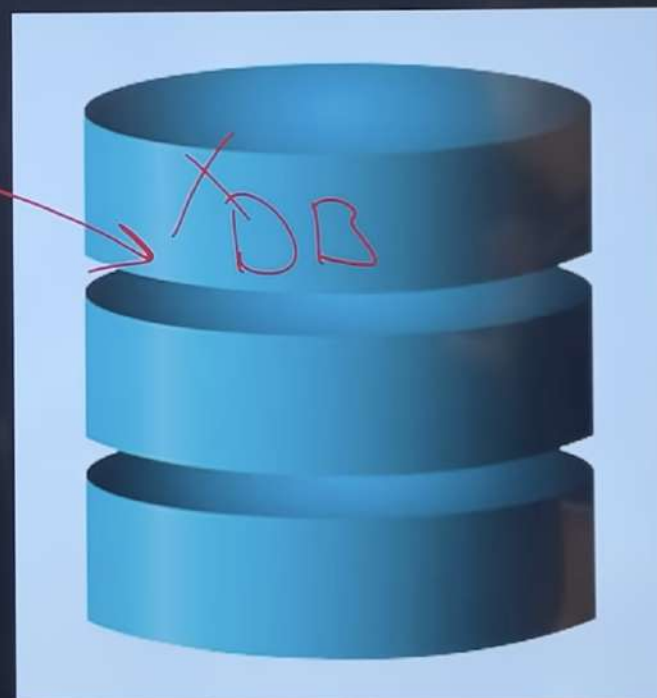
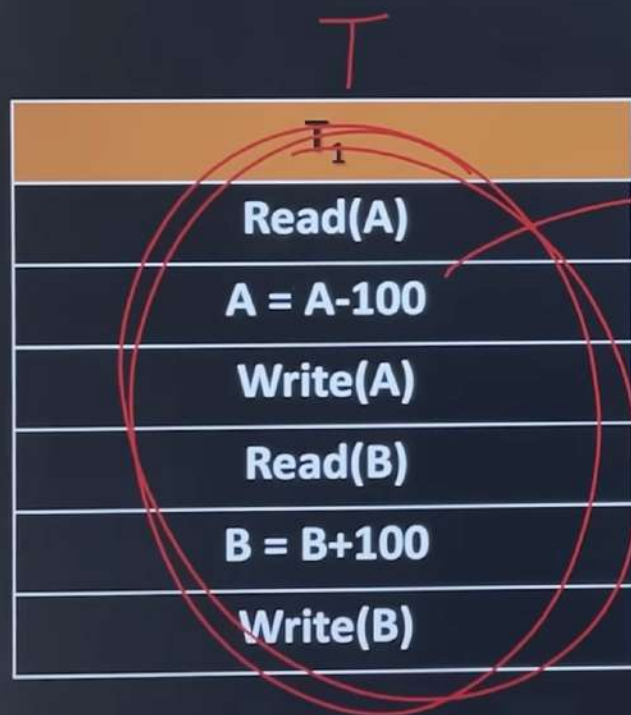
C = Consistency

I = Isolation

D = Durability

Desirable properties of transaction

- Now as the smallest unit which have atomicity in DBMS view is transaction, so if want that our data should be consistent then instead of concentrating on database, we must concentrate on the transaction for our data to be consistent.



Solution is Schedule

- When two or more transaction executed together or one after another then they can be bundled up into a higher unit of execution called schedule

T_0	T_1
read(A) write(A) read(B) write(B)	read(A) write(A) read(B) write(B)

T_2	T_3
read(B) read(A)	read(B) write(B) read(A) write(A)



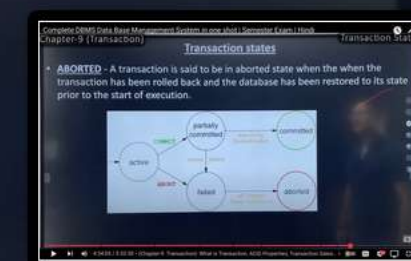
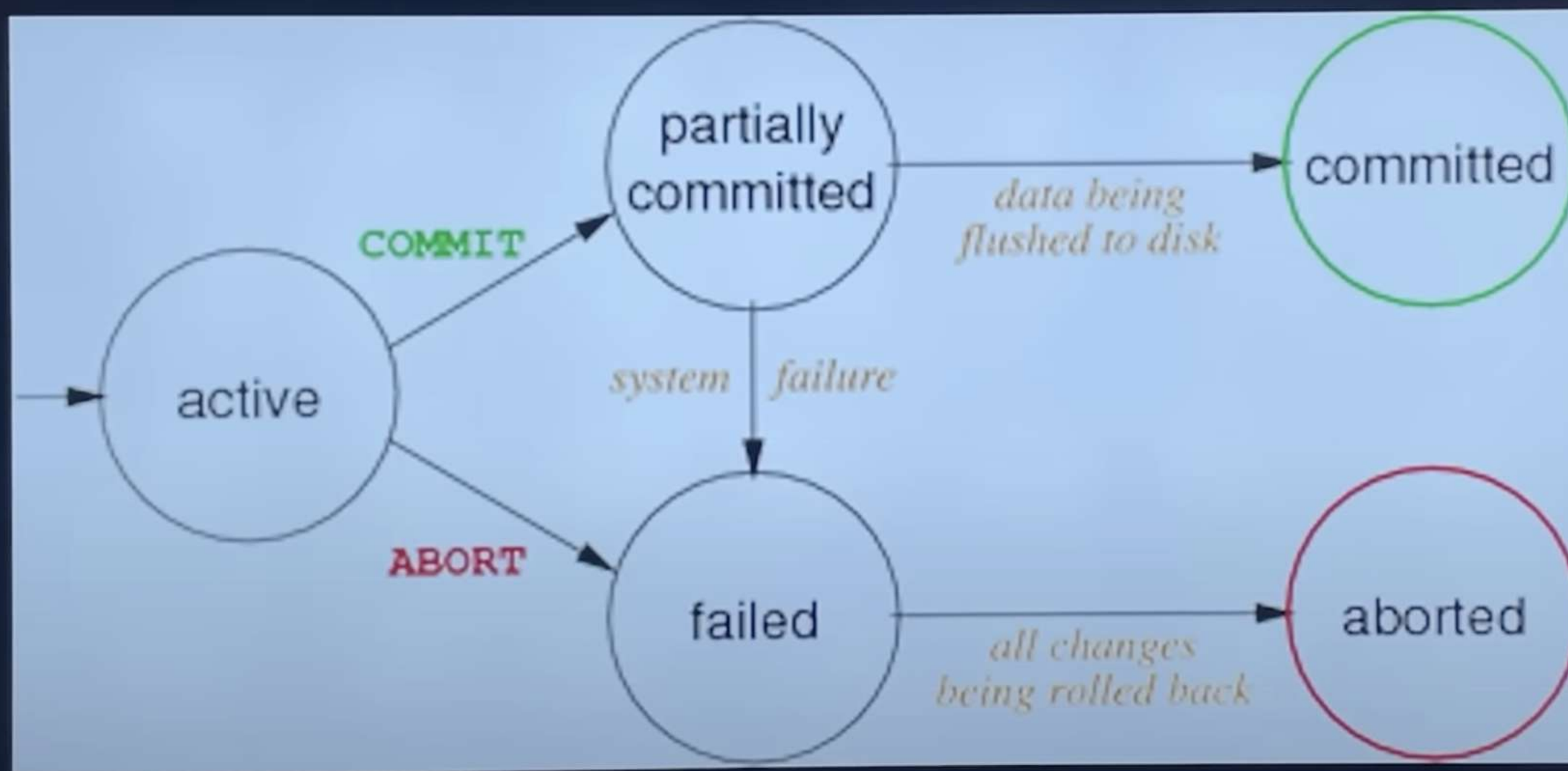
Why we need concurrent execution

- Concurrent execution is necessary because-
 - It leads to good database performance , less waiting time.
 - Overlapping I/O activity with CPU increases throughput and response time



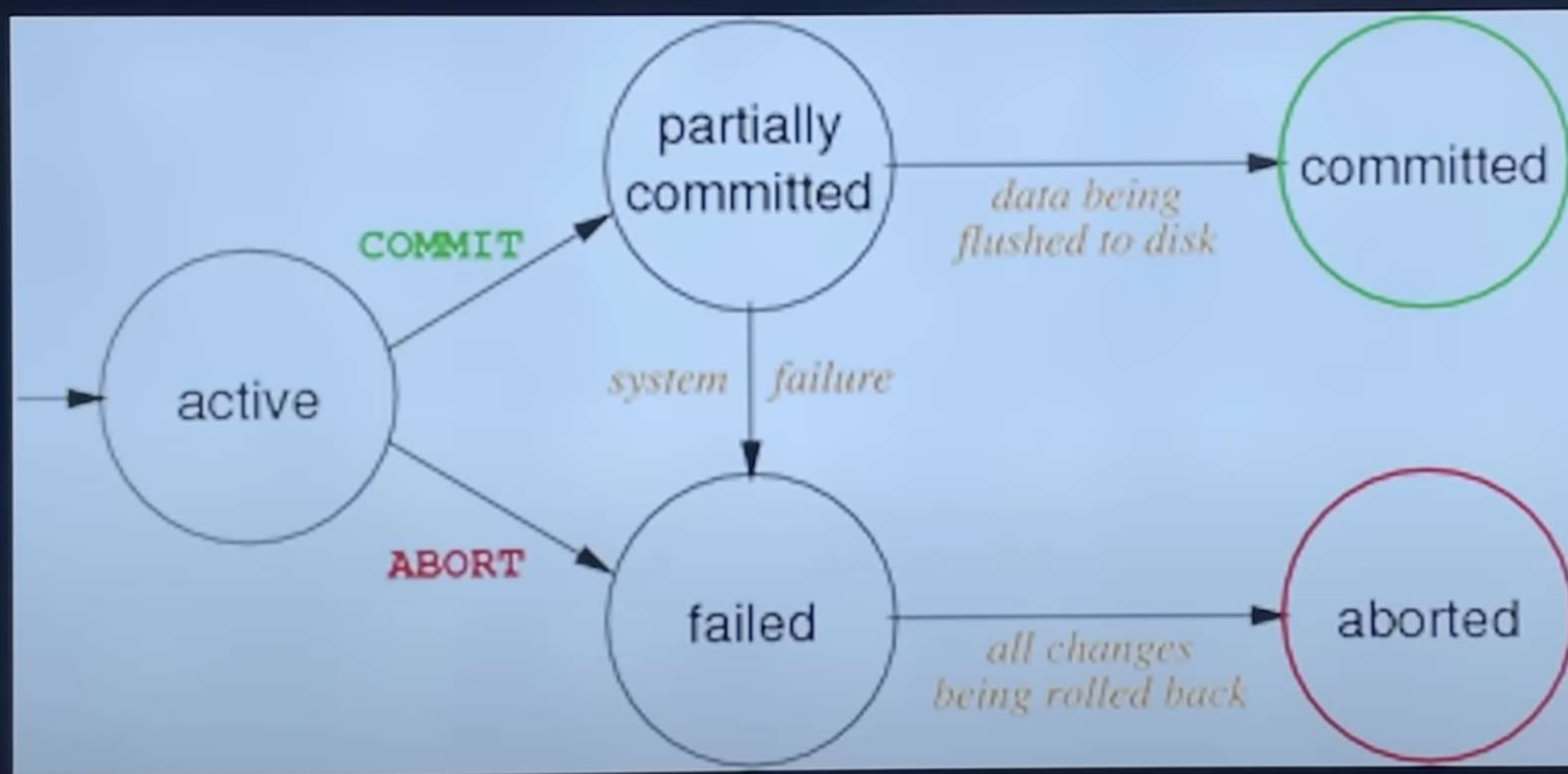
Transaction states

- **COMMITTED** - A transaction enters committed state after successful completion of a transaction and final updation in the database.



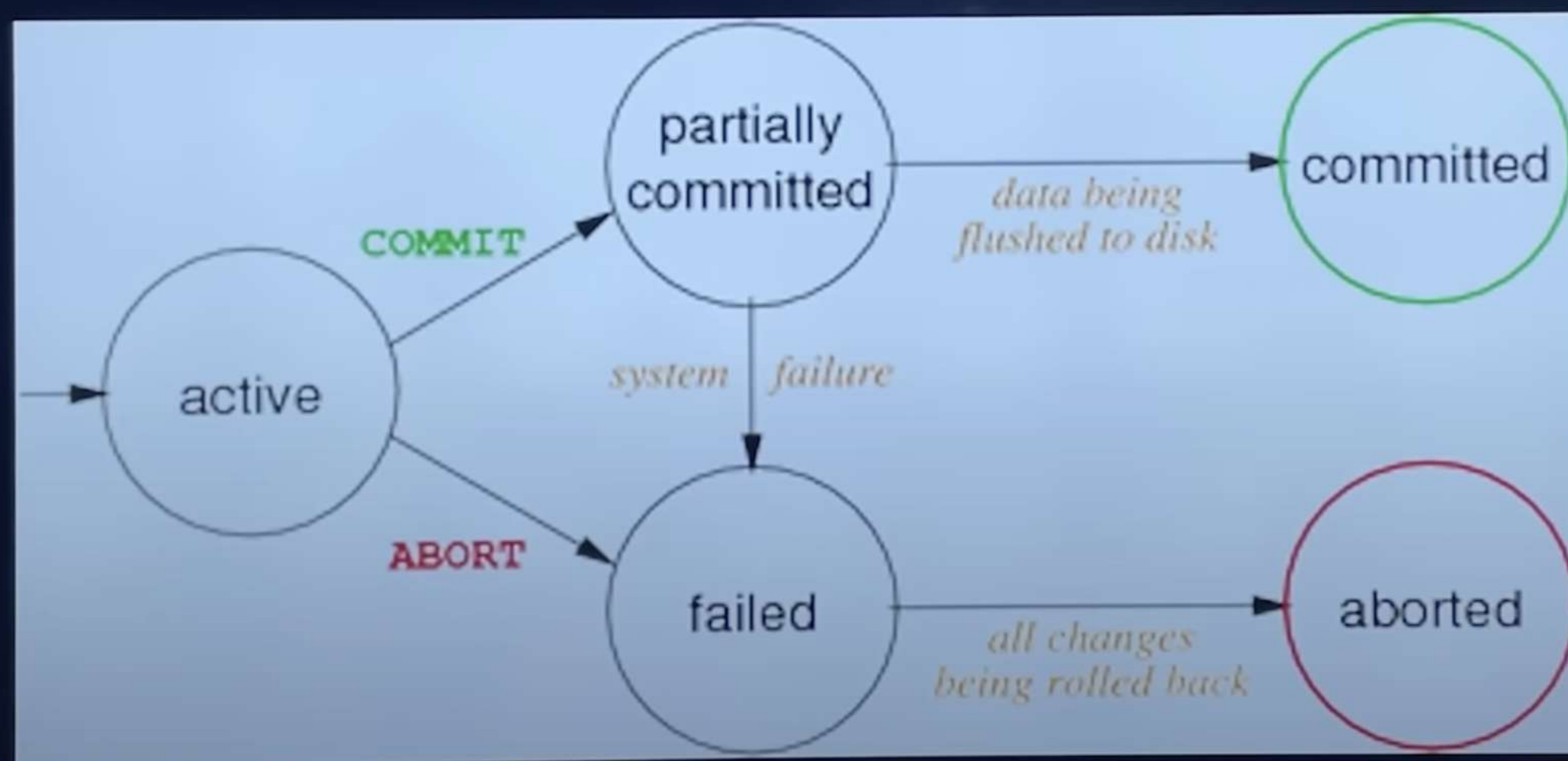
Transaction states

- **ABORTED** - A transaction is said to be in aborted state when the when the transaction has been rolled back and the database has been restored to its state prior to the start of execution.



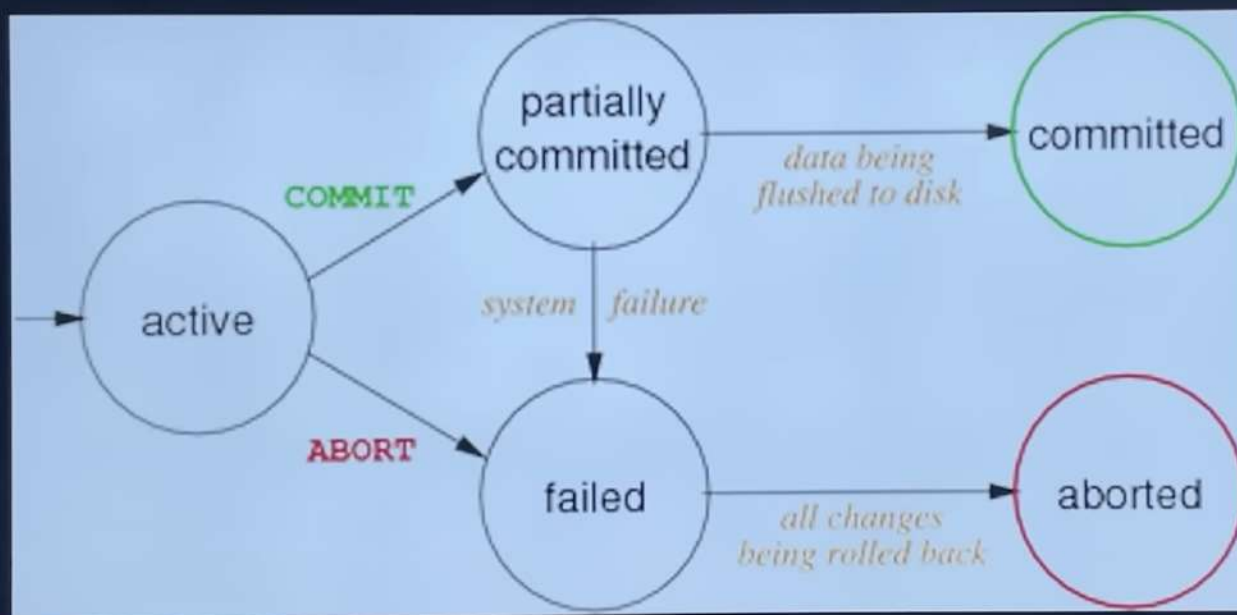
Transaction states

- **PARTIALLY COMMITTED** - After the final statement of a transaction has been executed, the state of transaction is partially committed as it is still possible that it may have to be aborted (due to any failure) since the actual output may still be temporarily residing in main memory and not to disk.



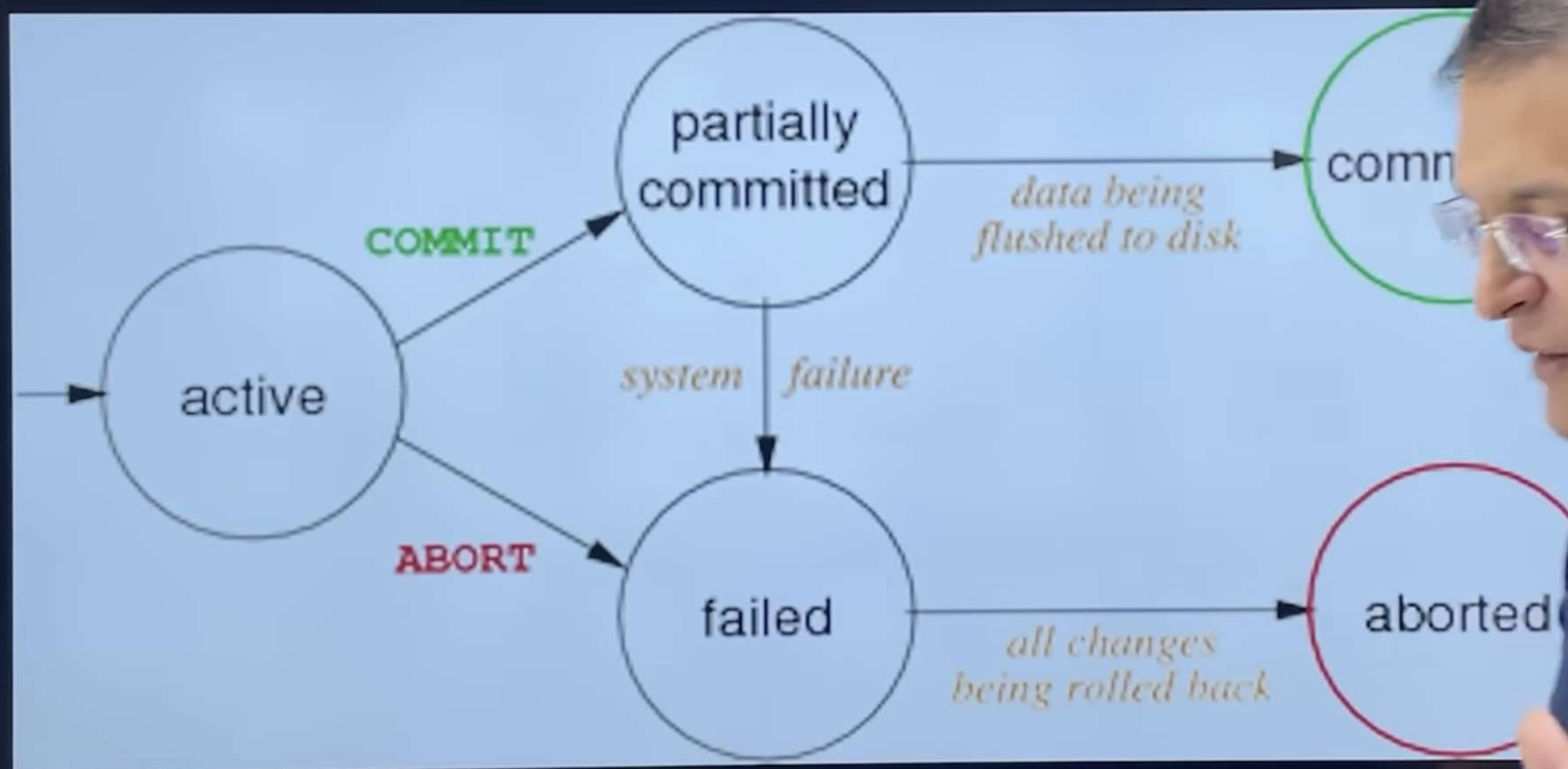
Transaction states

- **FAILED** - After the discovery that the transaction can no longer proceed (because of hardware /logical errors). Such a transaction must be rolled back.



Transaction states

- **ACTIVE** - It is the initial state. Transaction remains in this state while it is executing operations.

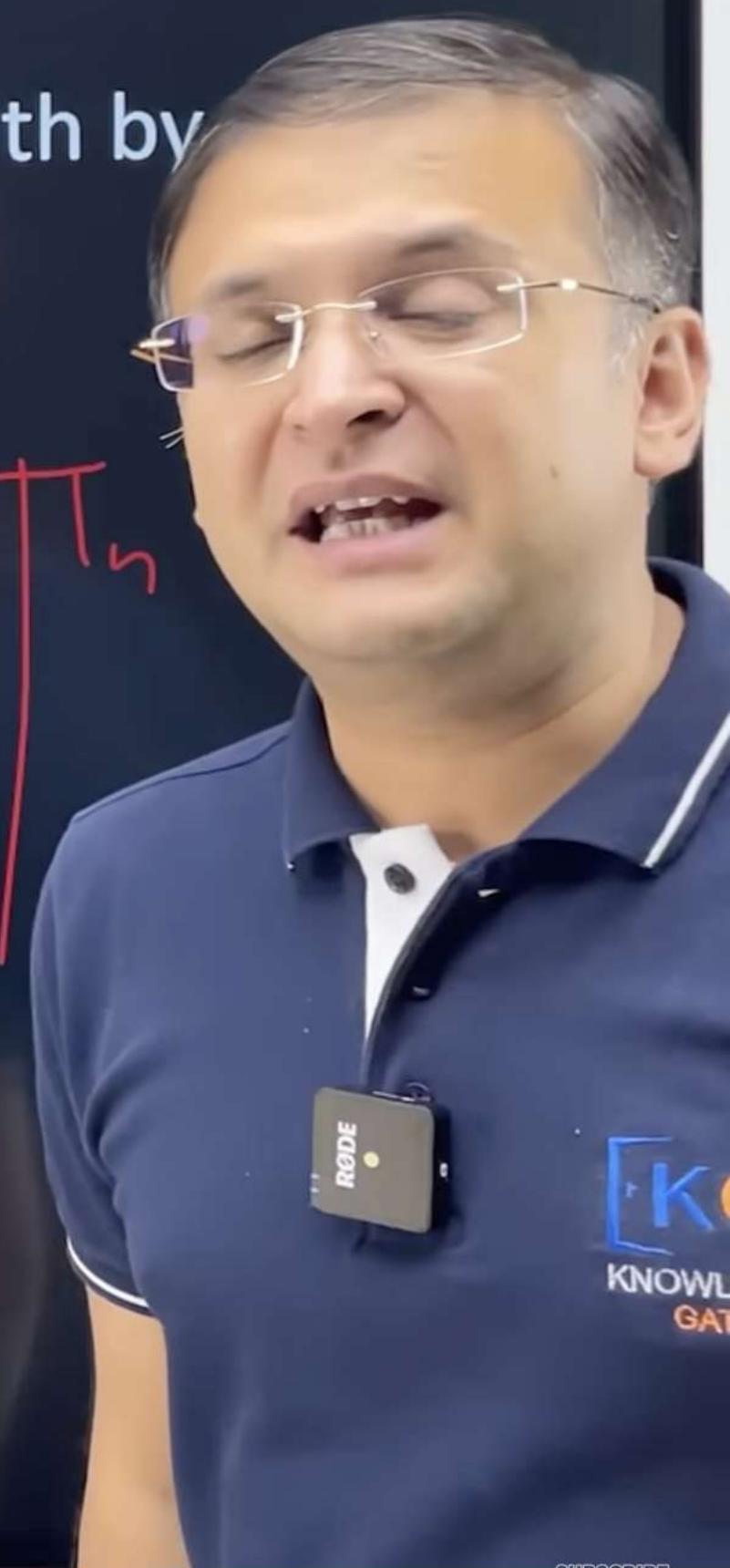


- **Durability** - The changes applied to the database by a committed transaction must persist in the database.



- Isolation - A transaction should appear as though it is being executed in isolation from other transactions, even though many transactions are executing concurrently.
- That is, the execution of a transaction should not be interfered with by transactions executing concurrently.

T_1 | T_2 | T_3 | T_n



- **Consistency** - A transaction should be consistency preserving, meaning that if it is completely executed from beginning to end without interference from other transactions, it should take the database from one consistent state to another.



Conflict equivalent – if one schedule can be converted to another schedule by swapping of non- conflicting instruction then they are called conflict equivalent schedule.

T_1	T_2
R(A)	
A=A-50	
	R(B)
	B=B+50
R(B)	
B=B+50	
	R(A)
	A=A+10

T_1	T_2
	R(B)
	B=B+50
R(A)	
A=A-50	
R(B)	
B=B+50	
	R(A)
	A=A+10

T ₁	T ₂
R(A)	
	R(B)

=

T ₁	T ₂
	R(B)
R(A)	

T ₁	T ₂
R(A)	
	W(A)

≠

T ₁	T ₂
	W(A)
R(A)	

T ₁	T ₂
R(A)	
	W(B)

=

T ₁	T ₂
	W(B)
R(A)	

T ₁	T ₂
	R(A)
W(A)	

≠

T ₁	T ₂
W(A)	
	R(A)

T ₁	T ₂
R(A)	
	R(A)

=

T ₁	T ₂
	R(A)
R(A)	

T ₁	T ₂
	W(A)
W(A)	

≠

T ₁	T ₂
W(A)	
	W(A)

NS → S

- **Conclusion of schedules**

- We do not have any method to proof that a schedule is consistent, but from the above discussion we understand that a serial schedule will always be consistent.
- So if somehow we proof that a non-serial schedule will also have same effects as of a serial schedule than we can proof that, this particular non-serial schedule will also be consistent.



Chapter-9 (Transaction)

- Non-serial schedule - A schedule in which sequence of instructions of a transaction appear in the same order as they appear in individual transaction but the instructions may be interleaved with the instructions of different transactions i.e. concurrent execution of transactions takes place.

T_2	T_3
read(B)	read(B) write(B)
read(A)	read(A) write(A)

Serial schedule - A serial schedule consists of sequence of instruction belonging to different transactions, where instructions belonging to one single transaction appear together. Before complete execution of one transaction another transaction cannot be started. Every serial schedule lead database into consistent state.

T_0	T_1
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)